LineTwin

A live discrete-event digital twin of a vehicle assembly line — real-time bottleneck detection validated against our own ground truth, defect-risk prediction with a stated honest-lift baseline, and graph-based inference at sensor-poor stations.

Built for the **Accenture Innovation Challenge 2026, Round 2, Problem Track 4 "DigitalTwin.ai"** as the Working Prototype deliverable. **184 tests passing locally.** A CI workflow for Ubuntu + Windows is committed; it has not run yet, and "CI green" is not claimed until it has.

> **Demo video** — attached to the latest release. The narration is `docs/VIDEO_NARRATION.md`; run the prototype yourself with `uv run python tools/demo.py`.
>
> **This README is also available as `README.pdf`** (`uv run --with markdown python tools/mkpdf.py README.md`).

Quickstart

```
git clone <this-repo> && cd linetwin
uv sync --all-extras
uv run python tools/generate_training_data.py   # simulates ~5 configs of labeled data; a few seconds, no network
uv run python tools/train_station_risk.py       # trains Model B (monotone logistic regression + Platt calibration)
uv run python tools/run_server.py             # or: tools/demo.py, which warms the line first
```

Then open **http://127.0.0.1:8000/**. One dashboard, four tabs — **Floor Supervisor**, **Plant Manager**, **Leadership**, and **Method** — all reading a single live SSE stream. The first three are the brief's three stakeholders; the navigation is the argument, not decoration.

Try the perturbation control on the Floor Supervisor tab (any station, any multiplier 0.1×–10×) and watch the constraint move on the line map, with the charts and alerts responding within ~2 seconds. Then open Plant Manager for the rolling-horizon forecast, the shifting-bottleneck timeline, and per-unit defect genealogy.

`uv run pytest -q` runs the full suite (**184 passing** once the two steps above have populated `ml/`; a few skip on a bare checkout until then; ~4 min). `uv run ruff check .` for lint.

For a demo, `uv run python tools/demo.py` audits readiness, warms the line, and prints a cue card — it refuses to call itself demo-ready if any committed result artefact is older than the simulation that produced it.

Scope, stated up front

LineTwin models **30 stations across three zones** (body construction, paint, final assembly) — a deliberately scoped subset of the 30–50 station line the brief describes. Station count, zones, cycle-time distributions, per-station instrumentation, condition drift and breakdown rates are all configuration (`scenarios/line30.yaml`). **Topology is not** — the line is a hardcoded serial chain, so parallel stations or rework loops would need code, not a new YAML file. Named rather than glossed; see `docs/SOLUTION_DESIGN.md` .

22 of 30 stations are instrumented; 8 are dark. The inference layer is load-bearing, not decorative — and that is *measured*, not asserted: it beats the trivial "use this station's zone base cycle time" estimator by **20–32% at every coverage level from 90% down to 40%** (`docs/phases/degradation_curve.csv`). That baseline arm did not exist until an audit added it, and when it did, it initially showed the layer performing **worse** than the baseline. The cause and the fix are recorded in `docs/phases/phase-09-sensor-gaps-genealogy.md` .

What this is, precisely

Under Kritzinger et al. (2018) this is a **Digital Shadow** — automated one-way data flow, no write-back to any physical system. Under Grieves & Vickers (2017) it is a **Digital Twin Prototype**: a virtual construct that exists before any physical instance, which is what the brief asks for. Under Villegas et al. (2025) it sits at the **Predictive** maturity level — one step short of prescriptive control action.

All simulation data is exactly that — simulated, from stated first principles, seed-reproducible, and labeled as model output at every layer. Parameters are calibrated against cited public sources where such sources exist, and stamped `synthetic – uncalibrated` where they do not. Full ledger: `docs/CITATIONS.md` .

Minimum signal required, per layer

Stated honestly because it is not the same answer for every layer, and the brief's uneven-sensor scenario deserves a real answer rather than one blanket claim.

Layer	Minimum signal needed	Typically already exists on an automotive line?	Does graph inference cover a gap here?
Flow / bottleneck detection	One bit per station: a photo-eye or PLC completion signal marking "a unit left this station, at this time."	Yes — virtually every automotive station already emits this. Starved-vs-blocked state is *derived* from adjacent buffer occupancy, not a separate sensor.	Not needed here — this signal is assumed present everywhere; the Active Period Method runs on it directly.
Quality / defect-risk	The station's actual cycle-time distribution and operational-state features (queue pressure, micro-stoppage rate, upstream risk).	No — this is where real instrumentation gaps genuinely bite; a manual-check station may report nothing between inspections.	**No.** Harmonic extension (Phase 9) infers a station's *cycle-time trend* from its neighbors — it does not and cannot infer whether a *specific unit* was defective. A missing quality signal is not recoverable by graph propagation, and we say so rather than imply otherwise.
Defect genealogy	The per-unit event log (already required by the flow layer) plus one defect flag from final inspection.	Yes, for the flow half; the inspection flag is the one new requirement, and it can arrive well downstream of the actual defect.	Not applicable — genealogy walks the existing per-unit log backward in time; it does not need to infer any missing per-unit signal, only realign timestamps by transfer delay.

Capability ladder

Every row names the test that actually proves it — not a description of intended behavior.

Capability	Proven by
A downstream slowdown propagates upstream as real blocking, not just a local slowdown	`tests/test_cascade.py`
Active-period counting matches the cited method exactly (a breakdown doesn't end an active period)	`tests/test_active_period.py`
Seven bottleneck detectors, scored against ground truth computed from our own sensitivity analysis	`tests/test_ground_truth.py`, `tests/test_bottleneck.py`, `tests/test_detectors.py`
The wire contract (`contracts.py`) is frozen and every fixture conforms to it	`tests/test_fixture_matches_contract.py`
The analytics path runs with `simpy` unimported — genuinely source-agnostic	`tests/test_source_agnostic.py`
Model B never trains and evaluates on the same line configuration	`tests/test_no_config_leak.py`
Model B's risk score is monotone in `cycle_time_z` across the full feature range	`tests/test_scorer.py`
Feature extraction samples live state correctly (no cumulative-delta drift)	`tests/test_features.py`, `tests/test_labels.py`
Model A reproduces a real dataset benchmark, including a deliberately-run SMOTE failure case	`tests/test_benchmark_public.py`
Rolling-horizon prediction produces a materially different forecast at a materially different horizon	`tests/test_rolling_horizon.py`
The two mandatory graph identities (exact mean at λ=0; exact partition of unity) hold numerically	`tests/test_inference.py`
Graph inference actually beats a no-graph baseline — not just that its algebra is correct	`tests/test_inference.py`
Unplanned stoppages occur, and a breakdown does not end an active period on a *running* line	`tests/test_active_period.py`
SPC control limits use mR-bar/d2, and the combined Western-Electric false-alarm rate is pinned	`tests/test_spc.py`
Greedy sensor placement only ever recommends currently-dark stations, and improves the rest	`tests/test_placement.py`
Defect genealogy correctly names an actually-injected origin station	`tests/test_genealogy.py`
The live engine's tick timing holds under real-time pacing, restart, and a background forecast task	`tests/test_engine.py`
Every REST route behaves correctly, including a live subprocess SSE stream	`tests/test_api.py`
The `ml` optional-dependency group is never imported by the server path	`tests/test_server_import_hygiene.py`

Capability	Proven by
ROI arithmetic is correct and non-negative across a realistic risk range	`tests/test_economics.py`

Measured results

Every number here is recomputed from a committed artefact by `uv run python tools/demo.py --check`, which refuses to call the prototype demo-ready if any of them was measured against a different simulation than the one that ships.

Bottleneck detection — seven methods, our own ground truth

30 scenario×seed trials (S05 / S17 / S25 engineered in turn, each scenario's ground truth independently re-verified at 60 replications before scoring).

Detector	Overall	S05	S17	S25
Busy Ratio	**90.0%**	100%	80%	90%
Active Period *(deployed)*	**86.7%**	100%	70%	90%
Turning Point	66.7%	100%	70%	30%
Queue Length	56.7%	90%	20%	60%
Arrow	56.7%	100%	70%	0%
Utilization	53.3%	100%	60%	0%

Busy Ratio beats the method we deploy, and the dashboard says so. The ranking is the finding: methods with real statistical structure survive correlated variation, while the point-statistic methods collapse to 0% on S25 — they compare single blocking/starving probabilities, which cannot separate a correlated regional slowdown from a genuine constraint.

Sensor-gap inference — measured against a baseline, not just an identity

Coverage	Graph inference	Zone-base baseline	Improvement
90%	0.052	0.078	**+32.5%**
70%	0.051	0.064	**+20.0%**
40%	0.050	0.067	**+26.2%**

Error stays between 4.8% and 5.2% as coverage falls from 90% to 40% — gradual, not a cliff.

The baseline arm did not exist until an audit added it, and when it did, the layer was initially 33–121% *worse* than the baseline. Correct linear algebra applied where its own precondition did not hold: harmonic extension

assumes smoothness over the graph, and the simulation was drawing every station independently. Fixed at the source, and a baseline-beating assertion is now a test — an identity test cannot tell you a method is useless. Full account: `docs/phases/phase-09-sensor-gaps-genealogy.md`.

Model B — defect risk, with its operating point

	Value	
PR-AUC (config E, never trained on)	**0.080**	9.5× the no-skill base rate
ROC-AUC	0.837	
Lift over `cycle_time_z` baseline	**+77.7%**	the single feature actually fitted for comparison
% of Bayes-optimal ceiling	97.5%	computable because we own the label process
Precision at threshold	**39.1%**	at the model's own MCC-tuned threshold (0.17)
Recall at threshold	2.9%	the deliberate trade — see below
False alarms per true catch	**~1.6**	

The brief warns that false alarms erode floor trust, so the operating point is on the dashboard, not just in a file. Low recall is the deliberate choice: the model stays quiet unless the signal is strong. And the label is linear by construction, so a linear model winning here is partly an artefact of the synthetic process — stated rather than claimed as a general result.

The honesty ledger

The discipline that makes the rest of this credible, summarized here and detailed in full in the linked documents:

- **`docs/CITATIONS.md`** — every verified source, thirteen individually-named figures we explicitly do *not* use, and every one of our own numbers with its mandatory qualification.
- **`docs/DATA.md`** — Model B's label-generating process, published *before* any metric was computed against it.
- **`docs/PRIOR_ART.md`** — our position on US 12,353,197 B2: what it *discloses*, what we adopt, how we differ, and the mandatory citation discipline (never "demonstrates").
- **`docs/LIMITATIONS.md`** — every known limitation in one place, including the ones a demo would usually prefer to leave out (Model B's features don't yet consume Phase 9's inferred values; the leadership ROI is an estimate, not an audited saving; operator variation is not modelled at all).
- **`docs/SOLUTION_DESIGN.md`** — the three brief areas answered by *design* rather than code (retrofit sensing tiers, the OT/PLC integration path, the maintenance-window rollout), each labelled as design and each stating what would have to be built.
- **`docs/adr/`** — four architecture decision records (simulation core, transport, ML data provenance, sensor gap), each stating the alternative considered and why it lost.

Integration and scalability

Integration. The twin is read-only with respect to any physical system it would sit beside — a `TelemetrySource` ABC (`src/twin/sources.py`) is the seam a real PLC/MES feed would implement; the simulator is one concrete implementation of it, proven interchangeable by `tests/test_source_agnostic.py` running the entire diagnostic and risk-scoring path with `simpy` itself unimported.

Scalability. Station count, zones, cycle-time distributions, buffer capacities, the instrumented/dark split, condition-drift parameters, breakdown profile and the sensor-gap operator weights are all one YAML file (`scenarios/line30.yaml`). A different line — more stations, a different zone mix, a different sensor-coverage ratio — is a new scenario file, not new code.

Topology is the exception, and it is named rather than glossed. The line is a hardcoded serial chain (`sim/line.py`) and the inference graph a hardcoded path (`graph/inference.py`), so parallel stations, rework loops and sub-assembly feeders would need code. The inference *maths* generalises to an arbitrary graph unchanged; the construction of that graph does not. See `docs/SOLUTION_DESIGN.md` .

Documentation

File	Contents
`docs/REQUIREMENTS.md`	Every brief requirement traced to a feature, module, and its evidence
`docs/CITATIONS.md`	Verified sources · individually-banned figures · our own numbers and their mandatory qualifications
`docs/DECISIONS.md`	Identity, architecture, scope cuts, contingency ladder
`docs/PRIOR_ART.md`	Position on US 12,353,197 B2 (Accenture) and mandatory citation discipline
`docs/DATA.md`	Model B's label-generating process, published before any metric
`docs/LIMITATIONS.md`	Every known limitation, named plainly, in one place
`docs/SOLUTION_DESIGN.md`	Retrofit sensing tiers · OT/PLC integration path · maintenance-window rollout — design, not code
`docs/adr/`	Four architecture decision records (ADR-003 partially superseded, marked)
`docs/phases/`	Per-phase build record, one markdown file per phase

Licence

Apache-2.0 — chosen over MIT for its explicit patent grant.